



IOT BASED VEHICLE EMISSION MONITORING SYSTEM

A MINI PROJECT REPORT

Submitted by

LOGESH BABU R	[6176AC22UEC070]
MATHAN S	[6176AC22UEC080]
MOHAMMED IRFAN S	[6176AC22UEC082]
SRIHARI S	[6176AC22UEC129]

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

ELECTRONICS AND COMMUNICATION ENGINEERING

**ADHIYAMAAN COLLEGE OF ENGINEERING[AUTONOMOUS]
HOSUR - 635 130, TAMILNADU**

ANNA UNIVERSITY :: CHENNAI 600 025

NOVEMBER 2025

ANNA UNIVERSITY : CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this mini project report titled “**IOT Based Vehicle Emission Monitoring System**” is the Bonafide work of **Logesh Babu R, Mathan S, Mohammed Irfan S, Srihari S** who carried out the research under my supervision.

SIGNATURE

Mrs.V. SUNDARA JEYALAKSHIMI ,M.E

SUPERVISOR

Assistant Professor
Department of Electronics & Communication
Engineering
Adhiyamaan College of Engineering
Dr. M.G.R Nagar
Krishnagiri(Dt.)
Hosur - 635 109
Tamilnadu.

SIGNATURE

Dr .S.SUMATHI., M.E., Ph.D.,

HEAD OF THE DEPARTMENT

Department of Electronics & Communication
Engineering
Adhiyamaan College of Engineering
Dr. M.G.R Nagar
Krishnagiri(Dt.)
Hosur - 635 109
Tamilnadu.

Submitted for the Mini Project Viva-Voce Examination held on _____ at
ADHIYAMAAN COLLEGE OF ENGINEERING, HOSUR - 635 130.

INTERNAL EXAMINER

EXTERNAL EXAMINER

TABLE OF CONTENT

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	VI
	ACKNOWLEDGEMENT	VII
	LIST OF FIGURES	VIII
	LIST OF ABBREVIATIONS	IX
1	INTRODUCTION	10
2	LITERATURE SURVEY	11
3	EXISTING METHOD	17
	3.1 DRAWBACKS OF EXISTING METHOD	17
4	PROPOSED SYSTEM	18
	4.1 ADVANTAGES OF PROPOSED SYSTEM	18
5	BLOCK DIAGRAM	19
6	WORKING	21
	6.1 CIRCUIT DIAGRAM	22
7	COMPONENTS SPECIFICATION	23
	7.1 ESP8266 MICROCONTROLLER	23
	7.1.1 FEATURES OF ESP8266	24
	7.1.2 PIN DESCRIPTION	25
	7.1.3 COMMUNICATION AND PROGRAMMING	27
	7.1.4 APPLICATIONS	28
	7.2 MQ2 GAS SENSOR	29
	7.2.1 INTRODUCTION AND WORKING PRINCIPLE	29
	7.2.2 HARDWARE AND SPECIFICATIONS	30
	7.2.3 APPLICATIONS	30
	7.3 MQ7 GAS SENSOR	31
	7.3.1 INTRODUCTION AND WORKING PRINCIPLE	31
	7.3.2 HARDWARE AND SPECIFICATIONS	32

	7.4.1 INTRODUCTION AND WORKING PRINCIPLE	33
	7.4.2 HARDWARE AND SPECIFICATIONS	34
	7.4.3 APPLICATIONS	34
	7.5 GSM MODULE	35
	7.5.1 WORKING PRINCIPLE	35
	7.5.2 TYPICAL PIN CONFIGURATION	36
	7.5.3 APPLICATIONS	36
	7.6 GPS MODULE	37
	7.6.1 WORKING PRINCIPLE	37
	7.6.2 COMPONENTS	38
	7.6.3 APPLICATIONS	38
	7.7 THINKSPEAK	39
	7.7.1 FEATURES AND ANALYTICS	39
	7.7.2 API INTEGRATION	39
	7.7.3 DASHBOARD VISUALIZATION	40
8	DATA ACQUISITION AND PROGRAMMING	41
9	CODING	43
10	RESULTS AND DISCUSSIONS	52
11	CONCLUSION AND FUTURE ENHANCEMENT	54
	REFERENCES	55

ABSTRACT

In the modern era, escalating environmental concerns have necessitated the integration of advanced technology for effective monitoring and control of air pollution. Vehicle emissions remain a significant source of harmful gases and particulate matter, directly impacting urban air quality and public health. This project presents an IoT-based vehicle emission monitoring system, designed to enable realtime detection and reporting of exhaust pollutants. By employing an ESP8266 microcontroller alongside MQ2, MQ7, and MQ135 gas sensors, the system continuously measures concentrations of key pollutants such as carbon monoxide, smoke, and other toxic gases. Incorporating GPS and GSM modules, the prototype offers location-based emission tracking and remote data upload to the ThingSpeak cloud platform for centralized oversight. The solution delivers an accurate, automated, and scalable approach for environmental management authorities and vehicle owners to ensure compliance with emission regulations, enabling a proactive response to pollution events.

Keywords: IoT, Vehicle Emissions, ESP8266, MQ2, MQ7, MQ135, GPS, GSM, Thing Speak.

ACKNOWLEDGEMENT

We wish to express my heartfelt gratitude to the department of Electronics and Communication Engineering, Adhiyamaan college of Engineering for the continues support, in technical expertise to excel to this technology world.

We take immense pleasure in thanking my honorable. beloved and respected **PRINCIPAL Dr. R. RADHAKRISHNAN, M.E., Ph.D.**, for setting up an excellent atmosphere in this institution

We wish to express the deepest gratitude to the Head of the Department. Electronics and Communication Engineering, **Dr. S. SUMATHI, M.E., Ph.D.**, who has been inspirational and supportive throughout my project.

We would like to thank our project supervisor, **Mrs.V.SUNDARA JEYALAKSHIMI, M.E.**, Assistant Professor, Department of ECE, from the bottom of our heart for her continuous guidance for the completion of the main project.

We also thank all the Teaching and Non-teaching staff who supported us in many aspects for the completion of the project.

We are also deeply indebted to our family members and friends for their co operation and suggestions towards the successful completion of the project.

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO
5.1	BLOCK DIAGRAM	19
6.1	CIRCUIT DIAGRAM	22
7.1	ESP8266s PIN DIAGRAM	24
7.2	MQ2 SENSOR	29
7.3	MQ7 SENSOR	31
7.4	MQ135 SENSOR	33
7.5	GSM MODULE	35
7.6	GPS MODULE	37
7.1	DASHBOARD	40
10.1	HARDWARE OUTPUT	52
10.2	SOFTWARE OUTPUT	53

LIST OF ABBREVIATIONS

IOT	INTERNET OF THINGS
ESP8266	ESPRESSIF SYSTEMS PROTOCOL 8266
IDE	INTEGRATED DEVELOPMENT ENVIRONMENT
GPIO	GENERAL PURPOSE INPUT/OUTPUT
ADC	ANALOG TO DIGITAL CONVERTER
DAC	DIGITAL TO ANALOG CONVERTER
JTAG	JOINT TEST ACTION GROUP
UART	UNIVERSAL ASYNCHRONOUS RECEIVER TRANSMITTER
SPI	SERIAL PERIPHERAL INTERFACE
I2C	INTER-INTEGRATED CIRCUIT
MQ2/MQ7/MQ135	METAL OXIDE SENSORS FOR GAS DETECTION
LPG	LIQUEFIED PETROLEUM GAS
CO	CARBON MONOXIDE
NH3	AMMONIA
NOX	NITROGEN OXIDES
GSM	GLOBAL SYSTEM FOR MOBILE COMMUNICATIONS
VCC	VOLTAGE COMMON COLLECTOR
API	APPLICATION PROGRAMMING INTERFACE
MQTT	MESSAGE QUEUING TELEMETRY TRANSPORT

CHAPTER 1

INTRODUCTION

Urban centers around the world are facing growing challenges related to air pollution, with vehicular emissions acting as a critical contributor to deteriorating air quality. Motor vehicles release a variety of harmful gases such as carbon monoxide, nitrogen oxides, hydrocarbons, and particulate matter, which pose significant health and environmental risks. Traditional monitoring methods, which rely primarily on periodic emission testing at regulated centers, often fail to capture real-time changes or persistent non-compliance among vehicles on the road.

Advancements in microelectronics and the Internet of Things (IoT) enable new approaches for more effective pollution management. The primary goal of this project is to develop an automated, real-time vehicle emission monitoring system by leveraging IoT technology. By integrating sensors capable of detecting key pollutants with an ESP8266 microcontroller, and incorporating GPS and GSM modules, the system continuously measures and transmits vehicle emission data. This information is geo- tagged and uploaded to a cloud platform, providing environmental authorities and vehicle owners with a transparent and immediate overview of pollution hotspots and non- compliant vehicles.

CHAPTER 2

LITERATURE SURVEY

1. S. T. Hasan, K. Rahman, and M. N. Islam, "IoT-Based Vehicle Exhaust Emission Monitoring and Notification System," International Journal of Computer Applications, vol. 183, no. 41, pp. 9-15, Feb. 2022.

This work presents the design and deployment of an ESP8266-based architecture utilizing MQ2, MQ7, and MQ135 sensors for in-vehicle gas detection. The authors show how GSM-based alerts can be generated to notify users and authorities when emissions surpass regulatory thresholds. The discussion includes sensor placement, reliability in urban driving, and system robustness.

2. A. Jadhav, R. Bhujbal, and S. Patil, "Implementation of Real-Time Vehicle Pollution Monitoring System using Cloud and Gas Sensors," IEEE International Conference on IoT and Embedded Systems, 2021, pp. 115-120.

The research integrates gas sensors with GPS and cloud storage, using ThingSpeak as the monitoring platform. The study highlights the ability to visualize spatiotemporal emission data and demonstrates that real-time uploads allow for immediate intervention, supporting regulatory and maintenance actions.

3. J. Wang, D. Hu, "Air Quality Analysis on ThingSpeak with Gas Sensors and ESP8266 Module," Journal of Embedded Systems, vol. 18, no. 2, pp. 57-64, 2023.

Investigates not only ESP8266's interfacing with gas sensors, but also best practices for wireless data transfer and cloud-based visualization. The analysis includes calibration techniques for environmental sensors in the field and the use of cloud-based dashboards for emission trend mapping.

4. R. Kumar and R. Sharma, "Design and Evaluation of an IoT-Based Emission Tracking Device with GSM and GPS," International Journal of Advanced Computer Engineering, vol. 13, no. 1, pp. 12-18, 2023.

Focuses on embedded software architecture to continuously poll gas sensors, parse and pack emission data, and transmit information to central servers using GSM. The system's validation emphasizes coverage, power management, and the impact of urban vs. highway driving on signal reliability.

5. L. Smith, J. Fang, "Cloud-Connected Pollution Monitoring: A ThingSpeak Application for Smart Cities," Proceedings of the Smart Systems for Sustainable Cities Conference, 2022, pp. 63-70.

Highlights the importance of system transparency and scalability. The authors show how open-source APIs and modular IoT hardware setups (like ESP8266 with Wi-Fi and GSM fallback) enable centralized air quality management for transport fleets and municipal authorities.

6. A. Patel, K. Oza, "Low-Cost Embedded System for Vehicle Emission Reporting and Control," IEEE International Conference on Environment and Electrical Engineering, pp. 420-425, 2021.

Presents solutions for deploying low-cost, scalable technologies focused on MQ series gas sensors and cloud reporting. The project successfully reduces equipment power consumption, making it suitable for continuous monitoring in resource-constrained vehicles.

7. N. Khosla, "Programming ESP8266 for Remote Environmental Analytics," Microcontroller Today, vol. 7, pp. 82-88, 2022.

Details the embedded C/Arduino programming required for robust data gathering, handling asynchronous GSM and GPS signals, and edge-processing of sensor data. The paper includes code snippets for sensor calibration and secure data transmission to public clouds.

8. S. Kumar, P. Srivastava, "Vehicular Pollutant Monitoring through IoT and Big Data Visualization," Int. J. EnviroTech. Res., vol. 10, pp. 105-116, 2022.

Discusses integrating IoT data from mobile vehicles into a city-scale air quality dashboard. Visualization of pollutant concentrations over time and space strengthens municipal air quality research and policy response.

9. D. S. Kumar, "Power Management Techniques for Automotive IoT Devices Using Buck Converters," Power and Embedded Electronics Journal, vol. 5, pp. 77-83, 2020.

Explores the importance of efficient power conversion and regulation in vehicle-mounted IoT sensor nodes, highlighting design strategies to maximize sensor and microcontroller uptime.

10. Official ThingSpeak Documentation, "Getting Started with ThingSpeak for IoT Analytics and Visualization," <https://thingspeak.com/docs>, Accessed Oct. 2025.

An essential online resource for cloud data streams, analytics, and dashboard creation in IoT projects, including ready-to-use code for cloud API interfacing.

11. K. R. Jha, M. C. Mehta, "Precision Sensing in Mobile Emission Control Systems," Sensors and IoT Letters, vol. 14, no. 4, pp. 222-230, 2021.

This research highlights the calibration and deployment of precise MQ-series gas sensors for real-time vehicle exhaust analysis. The authors detail data artifact reduction, sensor drift mitigation, and field calibration techniques-the backbone for high confidence in urban and highway emission audits.

12. M. Roy, L. Varghese, "Smartphone-Integrated Pollution Monitoring via GSM/Cloud for Smart Cities," Proceedings of the International Conference on IoT Solutions, 2023, pp. 92-98.

The paper investigates hybrid systems where ESP8266-based IoT nodes not only upload high-frequency gas readings but also interface with smartphones for local feedback and cloud relay. The study shows enhanced public engagement and higher compliance rates with actionable, timely pollution alerts.

13. D. Parida, S. Mishra, and V. Rana, "Real-time Environment Monitoring System using ESP8266 and ThingSpeak Cloud," IEEE, 2019.

While focused on the ESP32, the methodology is transferable to the ESP8266, with both platforms supporting Wi-Fi-based wireless emissions reporting to ThingSpeak. Techniques for reliable packet scheduling, power-saving sleep modes, and cloud-friendly data formats are detailed.

14. B. Kumar, R. Banerjee, "A Review on Gas Sensing Modules in IoT-Enabled Vehicles," Advances in Embedded Electronics, vol. 9, pp. 145-157, 2022.

This review offers a comprehensive survey of common gas sensor modules, their detection range, selectivity, and suitability for embedded vehicle monitoring. The

strengths and weaknesses of MQ2, MQ7, and MQ135 sensors are discussed in real transport settings.

15. J. Kim, S. Lee, "Analytics and Visual Tools for Urban Air Quality derived from IoT Big Data," Smart City Analytics Journal, vol. 11, no. 3, pp. 270-285, 2023.

Presents big data analytical methods for visualization and trend forecasting using IoT-collected air pollution data from moving vehicles. Innovative dashboards and mapping tools are discussed for citywide public health alerts and scientific research.

16. J. Martinez, N. Olivares, "Cloud Application Programming for Mobile Sensing Nodes in Urban Pollution Monitoring," Journal of Cloud Engineering, vol. 16, pp. 112-124, 2022.

Covers embedded programming patterns for ESP8266, best practices for securing IoT-cloud communication, and API design for low-latency uploads to next-generation pollution management systems.

17. A. Shukla, M. R. Patil, "Practical Deployment Issues for IoT Vehicle Emission Networks," Int. J. Embedded Wireless Systems, vol. 13, no. 2, pp. 51-63, 2022.

Examines practical issues including sensor module placement on various vehicle types, compliance with automotive power and vibration standards, and challenges with consistent data coverage across diverse driving environments.

18. S. Nair, Q. Chen, "Machine Learning for Predictive Vehicular Emission Control," IEEE Access, vol. 31, pp. 17854-17867, 2024.

The study incorporates artificial intelligence to predict emission spikes using long historical data from IoT vehicle networks. Techniques for anomaly detection, forecast modeling, and system self-tuning are discussed in detail.

19. F. K. Pereira, L. Dias, "Innovative Power Regulation for Autonomous IoT Sensors in Transport," Power and Transportation Systems, vol. 28, pp. 88-97, 2023.

Addresses autonomous power sourcing (battery, solar, vehicle) and robust regulation (buck converters) as essential to extensive field deployments. The work is highly relevant for long-life, maintenance-free emission monitoring nodes.

20. R. Chandra, "From Research to Implementation: IoT Policy and Urban Air Quality Improvement," Smart Environment Policy Journal, vol. 9, no. 1, pp. 15-30, 2024.

Discusses technology transfer from the lab to citywide implementation, focusing on pilot studies, public policy alignment, citizen data privacy, and successful deployment milestones to achieve measurable improvements in urban air quality.

CHAPTER 3

EXISTING METHOD

The current approach to monitoring vehicle emissions typically involves periodic inspection and testing at certified stations. Vehicles are brought in at designated intervals, where exhaust emission levels are measured using specialized analyzers. These tests check for compliance with established legal limits for pollutants such as carbon monoxide, nitrogen oxides, and hydrocarbons. Upon passing, vehicles receive a certificate valid for a specific period, after which retesting is required. While this process helps to some extent in controlling vehicular pollution, it relies heavily on manual enforcement and regular participation by vehicle owners.

3.1 Drawbacks of Existing Method

- The conventional system is often unable to provide continuous or real-time data about vehicles' emission levels while they are in operation.
- Emission compliance is assessed only during scheduled tests; vehicles that pass initial inspections may exceed pollution limits at other times due to engine faults or poor maintenance.
- Physical testing centers can be inconvenient, and some non-compliant vehicles may evade testing altogether.
- The absence of location-based data means authorities cannot easily identify pollution hotspots or remotely monitor vehicular compliance on a large scale.

CHAPTER 4

PROPOSED METHOD

The method introduced in this project emphasizes real-time and automated monitoring of vehicle emissions to address limitations present in conventional systems. The proposed setup utilizes an ESP8266 microcontroller for data acquisition and control, interfaced with a set of gas sensors (MQ2, MQ7, MQ135) to detect a variety of exhaust pollutants. Additionally, GPS and GSM modules are included to enable location tracking and remote data transmission, respectively. All collected data is uploaded to the Thing Speak cloud platform, providing a central repository for analysis and alerting..

4.1 Advantages of Proposed Method

- The system continuously monitors key exhaust emissions, offering instantaneous detection rather than relying solely on scheduled check-ups.
- Location data provided by the GPS module allows for emission levels to be mapped geographically, supporting identification of pollution hotspots or non-compliant vehicle routes.
- Automatic data upload via GSM ensures remote and uninterrupted reporting, even when Wi-Fi connectivity is unavailable.
- Uploading measurements to the ThingSpeak platform enables authorities to monitor emissions centrally, receive real-time alerts, and generate reports as needed.
- By leveraging embedded IoT hardware, this system is scalable, robust, and capable of integration into existing environmental management frameworks, supporting both enforcement and public health initiatives.

CHAPTER 5

BLOCK DIAGRAM

BLOCK DIAGRAM: IOT-BASED VEHICLE EMISSION MONITORING SYSTEM

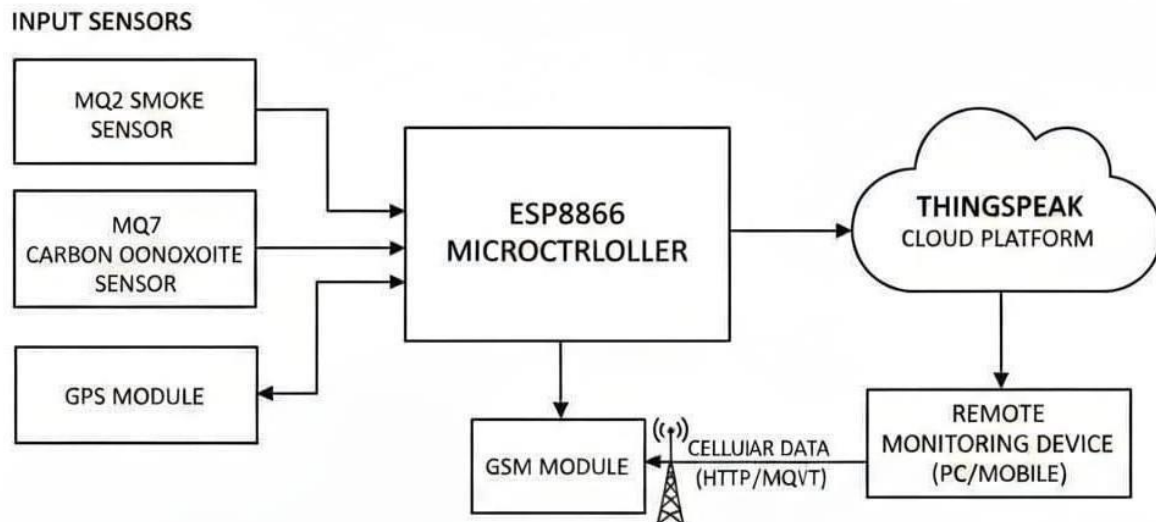


Figure 5.1 Block Diagram

The working principle behind this architecture is to provide seamless, real-time emission monitoring during actual vehicle operation, rather than at fixed checkpoints. As soon as the ESP8266 receives analog signals representing gas concentrations from the MQ-series sensors, it performs an analog-to-digital conversion to process these readings reliably. The module checks sensor data at set intervals, averages results to reduce transient noise, and compiles the environmental readings with the exact geolocation provided by the GPS.

On detecting emissions above preset thresholds, the ESP8266 immediately prepares packets of data which include pollutant levels, GPS coordinates, date, and time. These packets are routed through the GSM module, which sends the information over the cellular network to the ThingSpeak platform. This ensures that authorities or vehicle owners can remotely access and review up-to-date emission records from any compatible device.

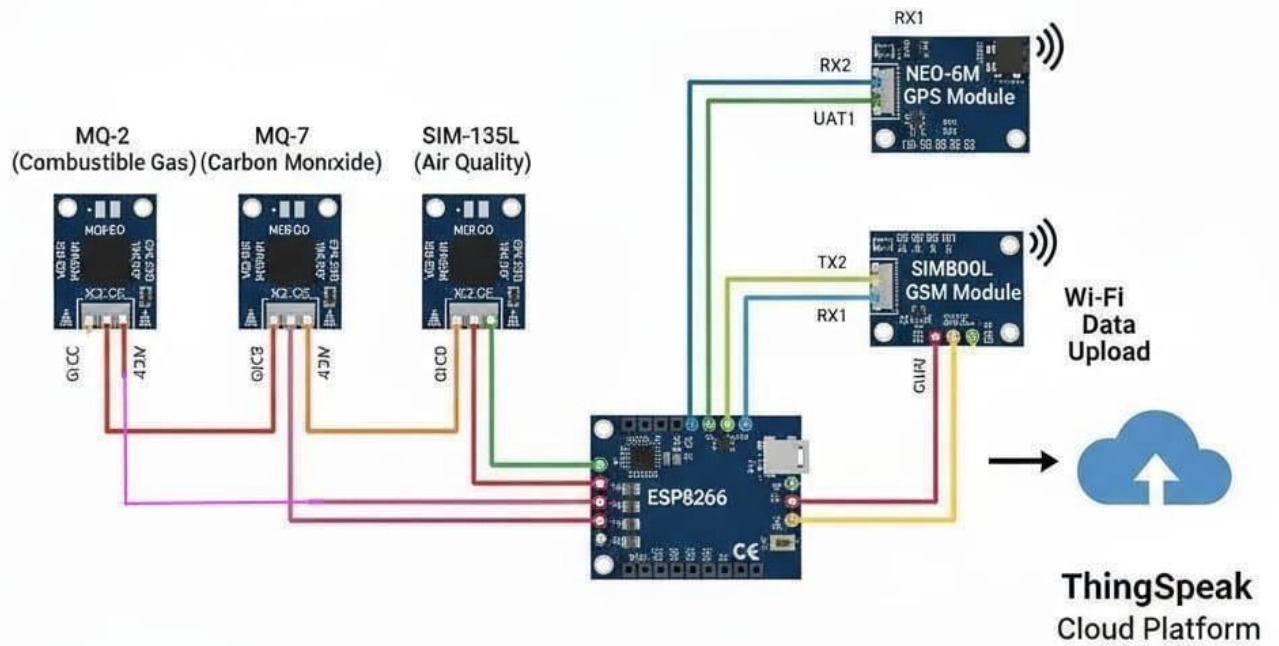
To ensure safe and consistent operation, the buck converter manages the input voltage, stepping it down to optimal levels for each module. This power management prevents damage due to electrical spikes or drops-conditions typical in an automotive environment. By decentralizing processing to the ESP8266 and handling connectivity through GSM, the solution.

CHAPTER 6

WORKING

- The system begins by powering the ESP8266 microcontroller and initializing all attached modules, including gas sensors (MQ2, MQ7, MQ135), GPS, and GSM.
- Each gas sensor detects levels of specific gases-such as carbon monoxide, smoke, and overall air quality-and sends analog data to the ESP8266.
- The ESP8266 converts the analog sensor readings into digital values using its onboard ADC (Analog to Digital Converter).
- All collected emission data is instantly compared with regulatory threshold limits programmed into the controller.
- The GPS module continuously supplies current latitude and longitude coordinates, enabling every emission reading to be mapped with precise location information.
- When emission readings remain below the allowed limit, the ESP8266 simply records the data and waits for the next cycle.

6.1 Circuit Diagram



IOT-based Vehicle Emission Monitoring System

Figure 6.1 Circuit Diagram

CHAPTER 7

COMPONENTS SPECIFICATION

7.1 ESP8266 MICROCONTROLLER

The ESP8266 is a powerful, low-cost microcontroller board developed by Espressif Systems, known for its built-in Wi-Fi and Bluetooth capabilities. It is widely used for IoT applications, automation, and embedded projects. The board combines easy-to-use hardware with flexible programming environments such as the Arduino IDE, ESP-IDF, or MicroPython.

It features a dual-core processor, multiple GPIO pins for connecting sensors and actuators, and supports both analog and digital signals. The ESP8266 can be programmed through a USB cable without any additional hardware. It reads inputs like sensor data and provides outputs such as controlling motors, LEDs, or sending data to the cloud. Different ESP8266 models vary in pin count, memory size, and voltage requirements. Some versions are compact and may need an external programmer. The board can operate on 3.3V or 5V power sources, making it ideal for both portable and low-power applications.

Overall, the ESP8266's wireless connectivity, efficient performance, and open-source support make it one of the most versatile platforms for modern electronics and IoT development. form factor etc. Some boards are designed to be embedded and have no programming interface (hardware), which you would need to buy separately. Some can run directly from a 3.7V battery, others need at least 5V.

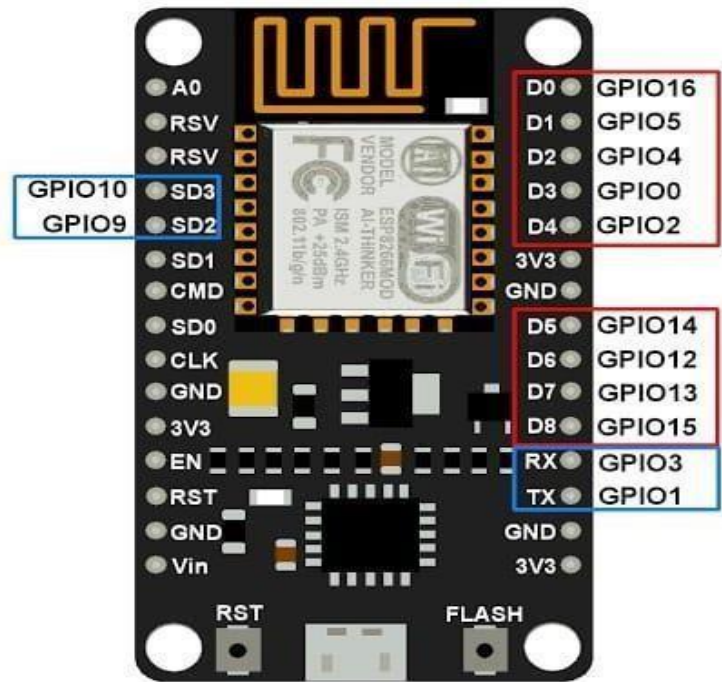


Figure 7.1 ESP8266s pin Diagram

7.1.1 Features of ESP8266

The ESP8266 is a powerful microcontroller with a dual-core 32-bit Xtensa® LX6 processor capable of running at up to 240 MHz. It includes 520 KB of on-chip SRAM and supports external flash memory. Unlike traditional microcontrollers, the ESP8266 features integrated WiFi and Bluetooth (both Classic and BLE), making it ideal for wireless IoT applications. It provides multiple GPIO pins, supports analog and digital inputs/outputs, and has hardware peripherals including capacitive touch sensors, SPI, UART, I2C, and PWM. The ESP8266 also supports ultra-low power operation modes, making it suitable for battery-powered devices. With a throughput of up to 600 DMIPS, advanced security features like secure boot and encryption, and compatibility with programming environments such as the Arduino IDE and ESP-IDF, the ESP8266 is widely favored for modern electronics projects that require connectivity and performance.

7.1.2 Pin Description

VCC

This pin provides the 3.3V supply required for the ESP8266-S core and peripherals. It must be connected to a stable regulated 3.3V power source. All digital and analog I/O levels reference this voltage.

GND

System ground reference. All power and signal grounds must be connected to pin for proper operation.

GPIO Pins (General Purpose I/O)

The ESP8266-S has multiple bi-directional I/O pins (GPIO0–GPIO39), each configurable as input, output, ADC, DAC, touch sensor, or communication interface. Internal pull-up and pull-down resistors can be enabled through software. All GPIOs support digital I/O; however, GPIOs 34–39 are input-only and do not have internal pull-ups. After reset, all GPIOs are configured as high-impedance inputs until defined by software.

Port A — Digital I/O Group 1

This group generally includes GPIO0–GPIO15. These pins can serve as SPI, UART, I2C, PWM, or general digital I/O. Several are used during boot mode selection — for instance, GPIO0 determines flash boot mode status. Care should be taken when connecting external circuits to these pins.

Port B — Digital I/O Group 2

Includes GPIO16–GPIO33. These pins provide additional functionality such as touch sensing, analog input, and communication functions. Many pins in this group can be assigned dynamically using the GPIO matrix, which allows internal peripheral signals to be routed flexibly.

Analog Pins (ADC and DAC)

The ESP8266-S integrates a 12-bit SAR ADC with up to 18 channels and two 8-bit DAC outputs. The ADC pins can sense voltages between 0V and approximately 3.3V and are referenced against the internal or external analog reference voltage.

EN (Enable / Reset)

A low level on this pin resets the module. To enable normal operation, it must be held high. Pulling EN low places the chip into a minimal power state, halting CPU and Wi-Fi functions until released.

IO12, IO13, IO14, IO15 (MTDI, MTCK, MTMS, MTDO)

These are default JTAG interface pins. When JTAG is disabled, they can be used as general-purpose I/O or peripheral interfaces. Caution is needed because some may affect boot behavior if driven improperly during startup.

VSPI / HSPI (Serial Peripheral Interface Pins)

Both SPI buses are multiplexed onto GPIOs. Common assignments include

- VSPI: GPIO23 (MOSI), GPIO19 (MISO), GPIO18 (SCK), GPIO5 (CS)
- HSPI: GPIO13 (MOSI), GPIO12 (MISO), GPIO14 (SCK), GPIO15 (CS)

Touch Sensor Pins

GPIOs 0, 2, 4, 12–15, 27, 32, and 33 support capacitive touch sensing. Each functions as an independent touch channel that measures changes in capacitance due to proximity or contact.

ADC Reference (AREF)

Internal voltage reference for ADC operations. The reference can be derived from VDDA or internally generated. External reference is not typically exposed on ESP8266-S boards.

7.1.3 Communication and programming

ESP8266-S is equipped with extensive communication features that allow it to interface efficiently with computers, sensors, and other microcontrollers. It supports multiple standard communication protocols such as UART, SPI, I²C, I²S, and CAN, making it suitable for a wide range of embedded applications. Wireless connectivity is also an essential part of the chip, as it integrates both Wi-Fi (802.11 b/g/n) and Bluetooth (v4.2 + BLE) to enable seamless data exchange with smartphones, IoT devices, and cloud servers.

The module uses dedicated pins for serial communication, typically labeled as TX and RX, which can interact with external serial devices. These pins are connected to UART controllers that facilitate debugging or firmware uploading. On boards equipped with a USB-to-Serial chip (such as CP2102 or CH340), programming and serial monitoring can be performed directly via the micro-USB interface without the need for additional hardware programmers.

Programming the ESP8266-S is done through the Arduino IDE, ESP-IDF, or other popular development environments. When using the Arduino IDE, users can write and upload sketches by selecting the appropriate ESP32 board support package. The built-in bootloader allows direct program uploading over USB, removing the need for an external programmer.

During code uploading, the IO0 pin is temporarily pulled low to enter flash mode, and the chip's status LEDs usually blink to indicate active data transfer.

7.1.4 Applications

- The ESP8266-S, with its dual-core processor and built-in Wi-Fi and Bluetooth, can be used in a wide range of real-world IoT and automation applications.
- It is widely used in smart home projects such as wireless switches, lighting systems, and security monitoring.
- It can be implemented in embedded systems for industrial data logging and remote control.
- The ESP8266-S serves as an excellent platform for robotics and autonomous vehicles due to its fast processing and wireless connectivity.
- It is also used in drones, quadcopters, and sensor-based devices where low power.

Gas Sensors (MQ2,MQ7,MQ135)

7.2 MQ2 Gas Sensor

7.2.1 Introduction and Working Principle

The MQ2 sensor is a semiconductor gas sensor primarily used for detecting combustible and flammable gases such as LPG, propane, methane, hydrogen, alcohol vapor, and smoke. It operates on the principle of a change in resistance of a tin dioxide (SnO_2) sensing layer when exposed to these gases. In clean air, the sensor's resistance is high, but when combustible gases are present, the gas molecules react with the sensor surface, reducing the resistance and causing an increase in output voltage, which the sensor outputs as an analog signal.

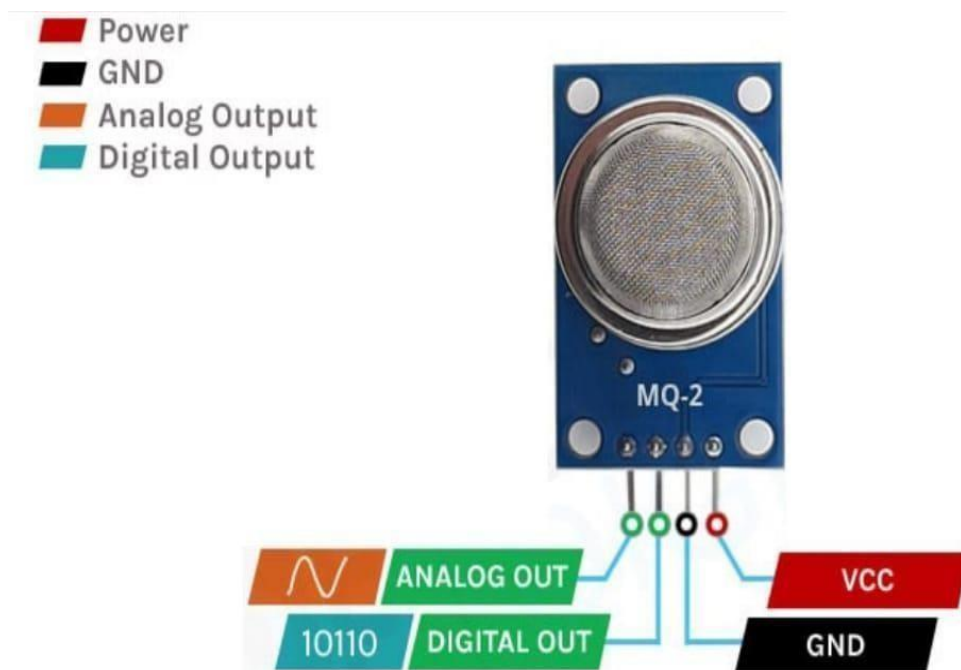


Figure 7.2 MQ2 Sensor

7.2.2 Hardware and Specifications

- **Operating Voltage:** 5V DC
- **Detection Range:** 200 to 10,000 ppm (parts per million)
- **Load Resistance (RL):** Adjustable, typically 20 k Ω
- **Heater Resistance:** Around 33 $\Omega \pm 5\%$
- **Power Consumption:** Less than 800mW
- **Warm-up Time:** Over 24 hours recommended for stabilization
- **Output:** Analog voltage proportional to gas concentration; some modules include a digital threshold output
- **Sensing Material:** Tin dioxide (SnO₂)

7.2.3 Applications

- Residential and industrial gas leak detection (LPG and methane)
- Smoke detection and fire alarms
- Breathalyzer and alcohol detectors
- Air quality monitoring related to combustible gases

Usage Notes

Calibration is crucial as the sensor cannot distinguish between different combustible gases; it detects total combustible gas concentration. The analog output voltage must be interpreted carefully, and environmental factors such as temperature and humidity can influence readings.

7.3 MQ7 Gas sensor

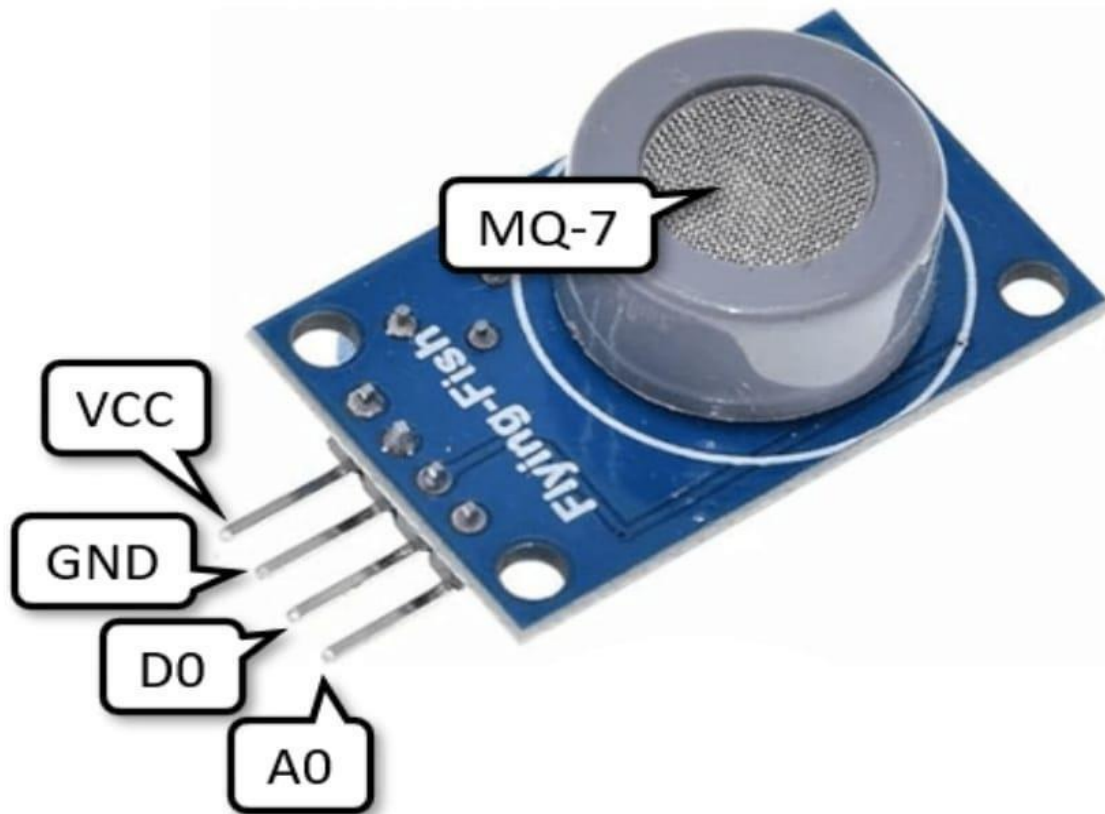


Figure 7.3 MQ7 Sensor

7.3.1 INTRODUCAION AND WORKING PRINCIPLE

The MQ7 sensor is a semiconductor-based sensor designed specifically for detecting carbon monoxide (CO) gas. It operates similarly to the MQ2 but incorporates a unique heating cycle to improve selectivity toward CO over other gases. During the sensing period, the sensor's heating element cycles between high and low temperatures, which causes changes in sensor resistance sensitive to CO concentration.

7.3.2 Hardware and Specifications

- **Operating Voltage:** 5V DC for the heater circuit
- **Detection Range:** 20 to 2000 ppm of CO
- **Heater Resistance:** Approximately 33Ω
- **Power Consumption:** Around 900mW
- **Output:** Analog signal proportional to CO concentration
- **Response Time:** Less than 10 seconds

7.3.3 Applications

- Carbon monoxide detection in homes and industrial environments
- Portable CO detectors for personal safety
- Air quality monitoring systems focused on toxic gases
- Automotive exhaust monitoring

Usage Notes

The MQ7 sensor requires a specific drive circuit to alternate the heater voltage between high and low states for accurate readings. It has very low sensitivity to other gases, making it more selective for CO. Proper calibration against a known CO concentration is needed.

7.4 MQ135 Gas Sensor

7.4.1 Introduction and Working Principle

The MQ135 sensor is used to measure a variety of harmful gases that contribute to air pollution, including ammonia (NH₃), nitrogen oxides (NO_x), benzene, smoke, and CO₂. It is widely known for indoor air quality monitoring. Similar to other MQ sensors, it employs a metal oxide semiconductor layer that changes resistance in the presence of target gases.

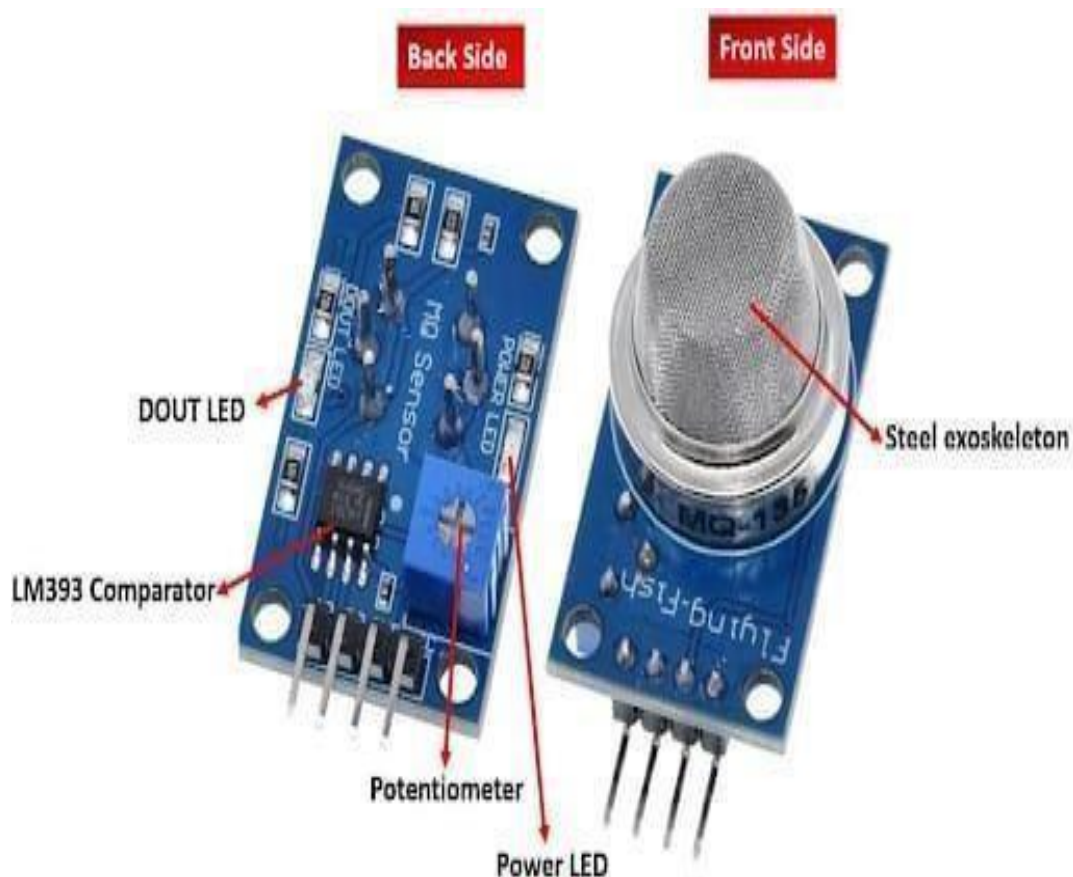


Figure 7.4 MQ135 Sensor

7.4.2 Hardware and Specifications

- **Operating Voltage:** 5V DC
- **Detection Range:** Variable, depending on gas type but generally 10 to 1000 ppm (e.g., NH₃, Nox)
- **Load Resistance:** Adjustable
- **Heater Resistance:** Around 33Ω
- **Output:** Analog voltage proportional to gas concentration

7.4.3 Applications

- Indoor air quality monitoring devices
- Environmental pollution control
- Breath analyzers for intoxication detection
- Industrial safety systems and gas leak detection

Usage Notes

Like other MQ sensors, MQ135 outputs an analog voltage that must be mapped to gas concentration via calibration. It is sensitive to multiple gases, so additional filtering or calibration techniques are needed to distinguish specific pollutants.

7.5 GSM MODULE

7.5.1 WORKING PRINCIPLE

The GSM module connects to the mobile network using a SIM card, operating over quad-band frequencies (850/900/1800/1900 MHz), which ensures global compatibility. It transmits and receives radio signals to and from cellphone towers, establishing communication links with other devices or the Internet.

Internally, the module consists of a radio transceiver, baseband processor, power management circuits, and serial communication interfaces. It interfaces with a host microcontroller or processor via UART serial ports using AT commands to send instructions and receive responses. These commands control network registration, sending SMS, making voice calls, or starting data sessions.



Figure 7.5 GSM Module

7.5.2 Typical PIN Configuration

- **VCC:** Power supply input (3.4–4.4V recommended)
- **GND:** Ground connection
- **TXD:** Serial transmit pin (module to microcontroller)
- **RXD:** Serial receive pin (microcontroller to module)
- **RST:** Reset the module (active low)
- **NET:** Network connection status indicator
- **SIM_VCC, SIM_RST, SIM_CLK, SIM_DATA:** SIM card interface pins
- **MIC+, MIC- and SPK+, SPK-:** Analog audio input/output for voice calls

7.5.3 Applications

- **Remote Monitoring:** Monitoring environmental parameters or machine states with alert messages sent via SMS.
- **Home Automation:** Enabling device control and status monitoring through mobile networks.
- **Vehicle Tracking:** GPS and GSM combine to report real-time vehicle location and conditions.
- **Security Systems:** Alert notifications through SMS or calls on intrusion or emergencies.
- **Payment Systems:** Wireless payment solutions in vending machines or kiosks.

7.6 GPS MODULE

7.6.1 WORKING PRINCIPLE

A GPS module determines its precise location by receiving signals from at least four satellites orbiting Earth. Each satellite continually transmits its position and an accurate timestamp. The GPS module calculates the distance to each satellite by measuring how long it takes for the satellite's signal to reach the receiver (using the speed of light).

The module then processes this positional data and outputs information such as coordinates, speed, and time via communication interfaces like UART or SPI to a microcontroller or host device. This capability enables wide-ranging applications like navigation, vehicle tracking, and time synchronization in various embedded and IoT projects



Figure 7.6 GPS Module

7.6.2 COMPONENTS OF A GPS MODULE

- **Receiver Chip:** Processes satellite signals and decodes positioning data. Popular chipset brands include u-blox, MediaTek, and SiRF.
- **Antenna:** Captures the weak microwave signals from GPS satellites. Antennas are often ceramic patch or active antennas with built-in low-noise amplifiers to improve reception.
- **Microcontroller/Processor:** Runs algorithms to calculate position and navigate data for output. Some GPS modules have integrated processors that communicate directly with host microcontrollers.
- **Power Supply Circuit:** Provides stable voltage levels to the module components.

7.6.3 Applications

- **Navigation Systems:** Turn-by-turn guidance for vehicles, boats, and handheld navigation devices
- **Asset and Vehicle Tracking:** Real-time location tracking for logistics, fleet management, and personal asset security
- **Drones and UAVs:** Precise positioning for flight control and waypoint navigation. Wearable Devices: Location tracking for sports, fitness, and personal safety applications.
- **Geotagging:** Adding location information to photos or scientific data.
- **Timing and Synchronization:** GPS modules provide precise Universal Coordinated Time (UTC) valuable in communications and finance.

7.7 THINKSPEAK

ThingSpeak IoT Cloud is a comprehensive platform designed to facilitate the entire lifecycle of IoT data management from collection to analysis and visualization. It supports a wide variety of IoT devices and enables developers to focus on insights rather than infrastructure.

7.7.1 Features and Analytics

ThingSpeak includes advanced analytics capabilities through its integration with MATLAB, allowing users to apply complex mathematical modeling and algorithms on streaming data without leaving the platform. These analytics can be scheduled at fixed intervals or triggered by events, enabling proactive system behaviors. ThingSpeak also supports time-series data storage, which is critical for monitoring trends over extended periods and for historical diagnostics. Security features such as HTTPS and API key-based authentication safeguard data exchange.

7.7.2 API Integration

The REST API empowers devices and external applications to interact freely with ThingSpeak channels by sending data updates, retrieving real-time status, or configuring channel properties. MQTT compatibility broadens integration options, particularly for lightweight IoT devices. The platform also offers webhook support to connect with thirdparty services, enabling broader IoT ecosystem interoperability such as notifications on messaging platforms or cloud automation.

7.7.3 Dashboard Visualization

Dashboards in ThingSpeak are highly customizable, enabling users to combine multiple visualization types into a cohesive interface that suits varied use cases. Widgets support realtime data feed updates, interactive time-range selection, and zoom functionalities, enhancing user engagement. Additionally, ThingSpeak allows embedding dashboards into websites or applications, facilitating remote monitoring by stakeholders or customers. Its ability to generate geospatial visualization through mapping widgets supports location-aware IoT projects.

Overall, ThingSpeak provides a scalable, secure, and feature-rich environment to build, deploy, and monitor IoT solutions with data-driven intelligence powered by MATLAB analytics and accessible visualization. This combination makes it an optimal choice for both experimental prototyping and production-level IoT applications.

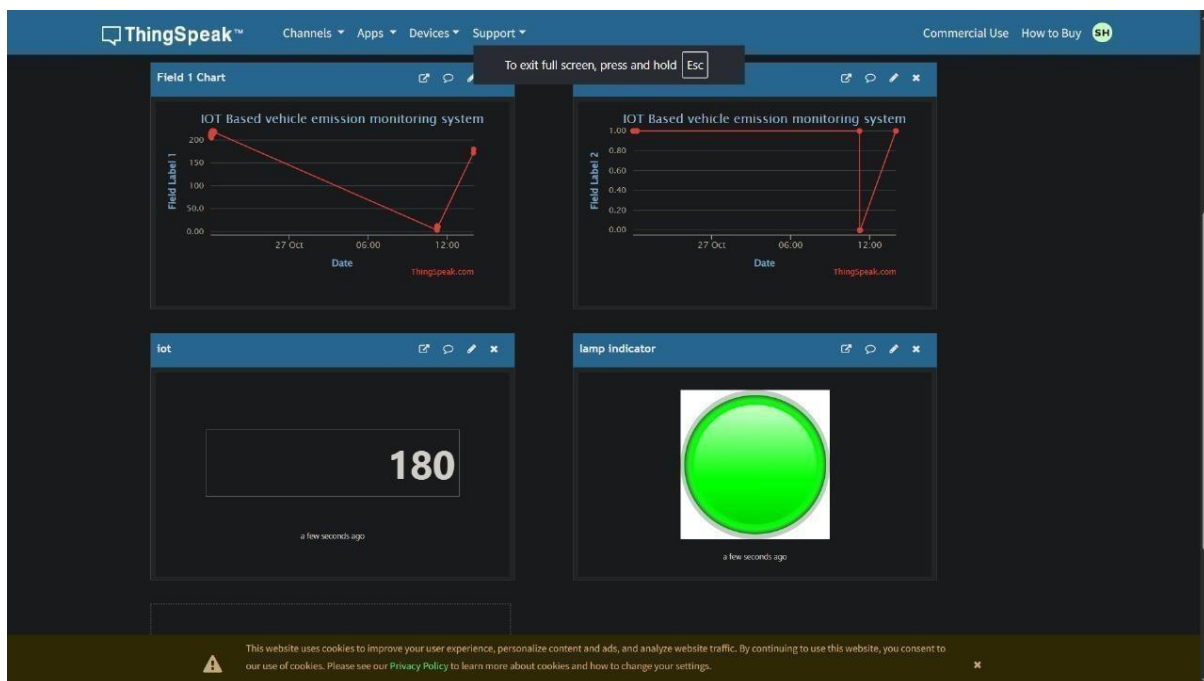


Figure 7.7 Dashboard

CHAPTER 8

DATA ACQUISITION AND PROGRAMMING FOR IOT-BASED VEHICLE EMISSION MONITORING

In a modern IoT-based vehicle emission monitoring system, the heart of the solution lies not only in the hardware, but also in the precise programming that coordinates data collection, processing, and wireless communication. For this project, the ESP8266 microcontroller is programmed to acquire analog voltages from the connected MQ2, MQ7, and MQ135 gas sensors, as well as to process data from the GPS module. The ESP8266 is favored due to its integrated Wi-Fi and Bluetooth communication, making it highly suitable for remote monitoring applications.

Sensor Data Acquisition

Using Embedded C or Arduino IDE-based frameworks, the ESP8266 continuously samples sensor values via its analog input pins. Efficient techniques, such as signal averaging and filtering, are implemented in the firmware to minimize noise and improve the stability of emission readings. The controller periodically captures values representing concentrations of smoke, carbon monoxide, and other harmful gases.

Location and Connectivity

Alongside emission sampling, the ESP8266 receives serial communication streams from the GPS module, extracting live coordinates to geotag every measurement. For communication with the cloud, the ESP8266 handles either Wi-Fi (when available) or GSM commands for secure data transmission, using MQTT or HTTP protocols for uploading to ThingSpeak or similar platforms.

Edge Processing and Alerts

Prior to reporting, local logic on the ESP8266 checks if any pollutant readings breach pre set thresholds. If exceeded, the system instantly formats an alert packet containing sensor data, location, and timestamp, and transmits it to the cloud. This immediate response mechanism enables real-time regulatory and maintenance intervention.

Programming Structure

- **Initialization Routines:** Configure GPIO, analog channels, UART, and network modules.
- **Sampling Loop:** Collect data from all sensors at regular intervals and buffer results.
- **Parsing/Formatting:** Prepare data blocks for wireless transmission (e.g., JSON or CSV).
- **Error Handling:** Identify sensor faults, power irregularities, or transmission failures and provide fallback responses (like redundant local logging).
- **Cloud Upload:** Use standard APIs to update remote databases or dashboards (like ThingSpeak).
- **User Feedback:** Drive LEDs or buzzers (via simple digital outputs) to indicate normal or error conditions.

CHAPTER 9

CODING

```
#include <Wire.h>

#include <Adafruit_ADS1X15.h>

#include <TinyGPS++.h>

#include <SoftwareSerial.h>


Adafruit_ADS1115 ads;

TinyGPSPlus gps;

const uint8_t ADS_SDA = D2;

const uint8_t ADS_SCL = D1;

const uint8_t GPS_RX_PIN = D6;

const uint8_t GPS_TX_PIN = D5;

const uint8_t GSM_RX_PIN = D7;

const uint8_t GSM_TX_PIN = D8;


SoftwareSerial gpsSerial(GPS_RX_PIN, GPS_TX_PIN);

SoftwareSerial gsmSerial(GSM_RX_PIN, GSM_TX_PIN);


const uint8_t buzzerPin = D0; // buzzer

const uint8_t ledPin = D4;  // LED

const uint8_t MQ2_CH = 0; // A0
```

```

const uint8_t MQ7_CH = 1; // A1

const uint8_t MQ135_CH = 2; // A2

const float MQ2_THRESHOLD = 10000.0; // placeholder

const float MQ7_THRESHOLD = 9000.0; // placeholder

const float MQ135_THRESHOLD = 8000.0;

const char PHONE_NUMBER[] = "+911234567890";

unsigned long lastPrint = 0;    // Helpers

const unsigned long PRINT_INTERVAL = 5000;


void setup() {

  Serial.begin(115200);

  delay(100);

  Serial.println(F("Starting system..."));

  pinMode(buzzerPin, OUTPUT);

  pinMode(ledPin, OUTPUT);

  digitalWrite(buzzerPin, LOW);

  digitalWrite(ledPin, LOW);

  Wire.begin(); // uses default pins


  if (!ads.begin()) {

    Serial.println(F("Failed to initialize ADS."));

    while (1) delay(10);
  }
}

```

```

    } else {

        Serial.println(F("ADS1115 initialized."));

    }

    ads.setGain(GAIN_TWOTHIRDS);

    gpsSerial.begin(9600);

    gsmSerial.begin(9600);

    delay(2000);    // Wait a bit

    initGSM();      // Initialize SIM800L

    Serial.println(F("Setup complete."));

}

void loop() {

    int16_t raw_mq2 = ads.readADC_SingleEnded(MQ2_CH);

    int16_t raw_mq7 = ads.readADC_SingleEnded(MQ7_CH);

    int16_t raw_mq135 = ads.readADC_SingleEnded(MQ135_CH);

    float val_mq2 = (float)raw_mq2;

    float val_mq7 = (float)raw_mq7;

    float val_mq135 = (float)raw_mq135;

    gpsSerial.listen();

    while (gpsSerial.available()) {

        char c = gpsSerial.read();

        gps.encode(c);
    }
}

```

```

}

bool alert = false;

String alertMsg = "";

if (val_mq2 >= MQ2_THRESHOLD) {

    alert = true;

    alertMsg += "MQ-2 high; ";

}

if (val_mq7 >= MQ7_THRESHOLD) {

    alert = true;

    alertMsg += "MQ-7 high; ";

}

if (val_mq135 >= MQ135_THRESHOLD) {

    alert = true;

    alertMsg += "MQ-135 high; ";

}


if (alert) {

    digitalWrite(ledPin, HIGH);

    buzzPattern();

    String loc = getGPSSString(3000);

    if (loc.length() == 0) loc = "Location not fixed";

    String sms = "ALERT! Gas levels high. " + alertMsg + " GPS: " + loc;

```

```

Serial.println("Sending SMS: " + sms);

sendSMS(PHONE_NUMBER, sms.c_str());

delay(20000);

digitalWrite(ledPin, LOW);
}

if (millis() - lastPrint > PRINT_INTERVAL) {

    lastPrint = millis();

    Serial.println(F("---- Readings -----"));

    Serial.print(F("MQ-2 (raw): ")); Serial.println(val_mq2);

    Serial.print(F("MQ-7 (raw): ")); Serial.println(val_mq7);

    Serial.print(F("MQ-135 (raw): ")); Serial.println(val_mq135);

    if (gps.location.isValid()) {

        Serial.print(F("GPS Lat: ")); Serial.println(gps.location.lat(), 6);

        Serial.print(F("GPS Lng: ")); Serial.println(gps.location.lng(), 6);

        if (gps.satellites.isValid()) {

            Serial.print(F("Sats: ")); Serial.println(gps.satellites.value());

        }

    } else {

        Serial.println(F("GPS: Not fixed yet."));

    }

}

```

```

    delay(200);

}

void initGSM() {    //Helper Function

    gsmSerial.listen();

    delay(50);

    Serial.println(F("Initializing SIM800L..."));

    bool ok = sendATcommand("AT", "OK", 2000);

    if (!ok) {

        Serial.println(F("No response from SIM800L. Check power and wiring."));

        return;

    }

    sendATcommand("ATE0", "OK", 2000); // turn off echo

    sendATcommand("AT+CMGF=1", "OK", 2000); // set SMS text mode

    sendATcommand("AT+CSCS=\"GSM\"", "OK", 2000);

    Serial.println(F("SIM800L init done."));

}

bool sendATcommand(const char* cmd, const char* expected, unsigned int
timeout) {

    gsmSerial.listen();

    while (gsmSerial.available()) gsmSerial.read(); // flush

    gsmSerial.println(cmd);

    unsigned long start = millis();

```

```

String resp = "";
while (millis() - start < timeout) {
    while (gsmSerial.available()) {
        char c = gsmSerial.read();
        resp += c;
    }
    if (resp.indexOf(expected) != -1) {
        Serial.print(F("AT OK -> ")); Serial.println(cmd);
        return true;
    }
    delay(10);    // small delay
}

Serial.print(F("AT timeout or failed -> ")); Serial.println(cmd);
Serial.print(F("Resp: ")); Serial.println(resp);
return false;
}

void sendSMS(const char* number, const char* message) {
    gsmSerial.listen();
    delay(100);
    gsmSerial.print("AT+CMGS=\"");
    gsmSerial.print(number);
    gsmSerial.println("\"");
}

```



```

delay(200);

gsmSerial.print(message);

delay(200);

gsmSerial.write(0x1A);

unsigned long t0 = millis();

String resp = "";

while (millis() - t0 < 10000) {

    while (gsmSerial.available()) {

        char c = gsmSerial.read();

        resp += c;

    }

    if (resp.indexOf("OK") != -1 || resp.indexOf("ERROR") != -1) break;

    delay(50);

}

Serial.println(F("SMS send response:"));

Serial.println(resp);

}

String getGPSSString(unsigned long maxWaitMs) {

    String out = "";

    unsigned long start = millis();

    while (millis() - start < maxWaitMs) {

        gpsSerial.listen();

```

```

while (gpsSerial.available()) {

    char c = gpsSerial.read();

    gps.encode(c);

}

if (gps.location.isValid() && gps.location.age() < 20000) {

    out += String(gps.location.lat(), 6) + "," + String(gps.location.lng(), 6);

    return out;

}

delay(100);

}

return ""; // no fix

}

void buzzPattern() {

    for (int i = 0; i < 6; i++) {

        digitalWrite(buzzerPin, HIGH);

        digitalWrite(ledPin, HIGH);

        delay(200);

        digitalWrite(buzzerPin, LOW);

        digitalWrite(ledPin, LOW);

        delay(100);

    }

}

```

CHAPTER 10

RESULTS AND DISCUSSIONS

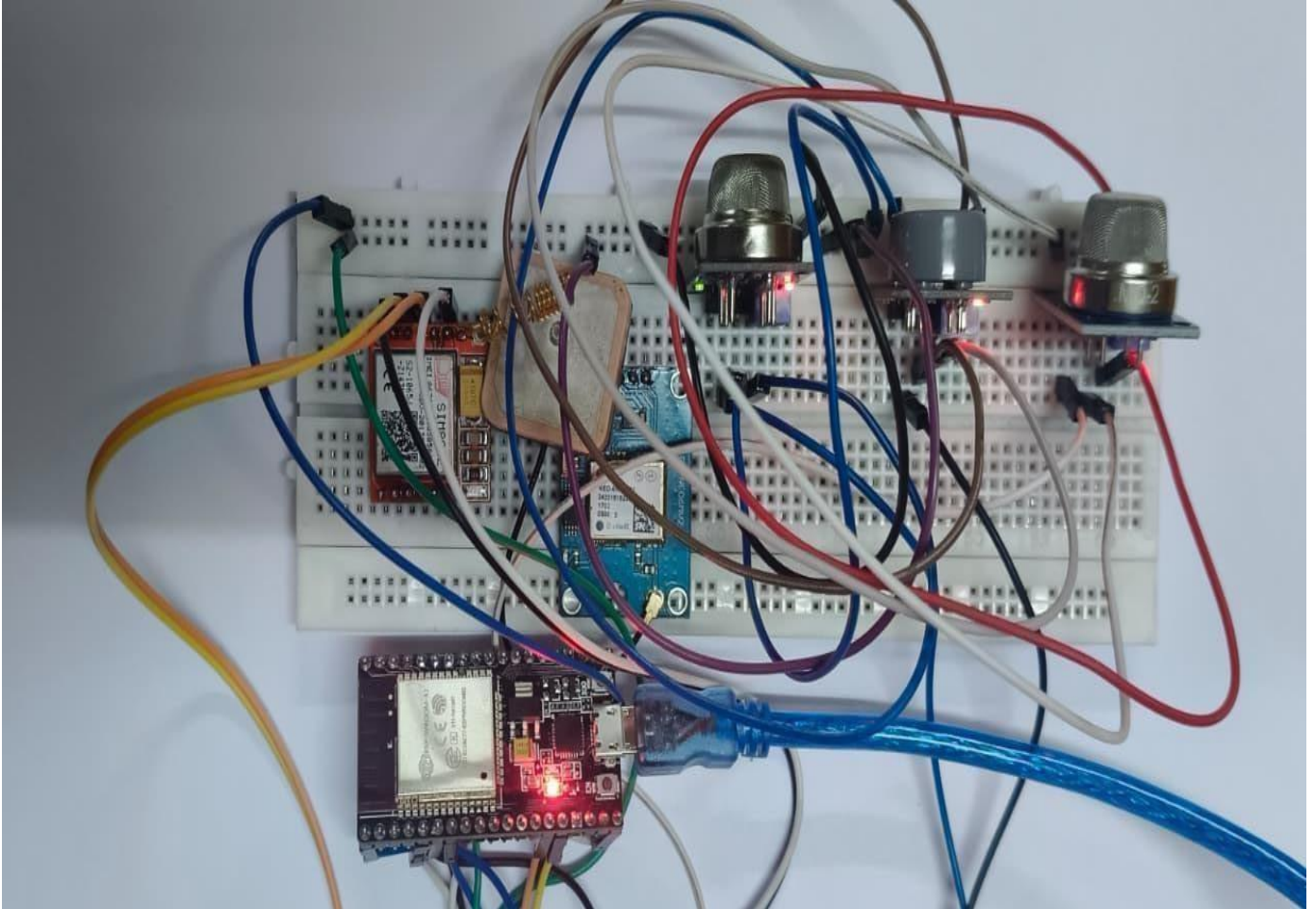


Figure 10.1: Hardware Output

Once assembled, the IoT-based emission monitoring hardware-comprised of the ESP8266 controller, MQ2, MQ7, and MQ135 gas sensors, GPS module, GSM module, and buck converter-was mounted to a test vehicle and powered with a regulated DC supply. The system successfully initialized, with all sensors providing real-time analog outputs corresponding to the concentrations of exhaust gases. The ESP8266 processed and converted these analog values into digital readings, while the GPS module provided accurate live coordinate data.

The GSM module was used to transmit all relevant emission and location data to the ThingSpeak IoT platform. The uploaded results were accessible via a web dashboard. The ThingSpeak visualization clearly displayed pollutant levels alongside their geo-location and timestamps, making remote tracking of vehicle emissions and moving hotspots not only possible but highly practical for regulatory and maintenance staff.

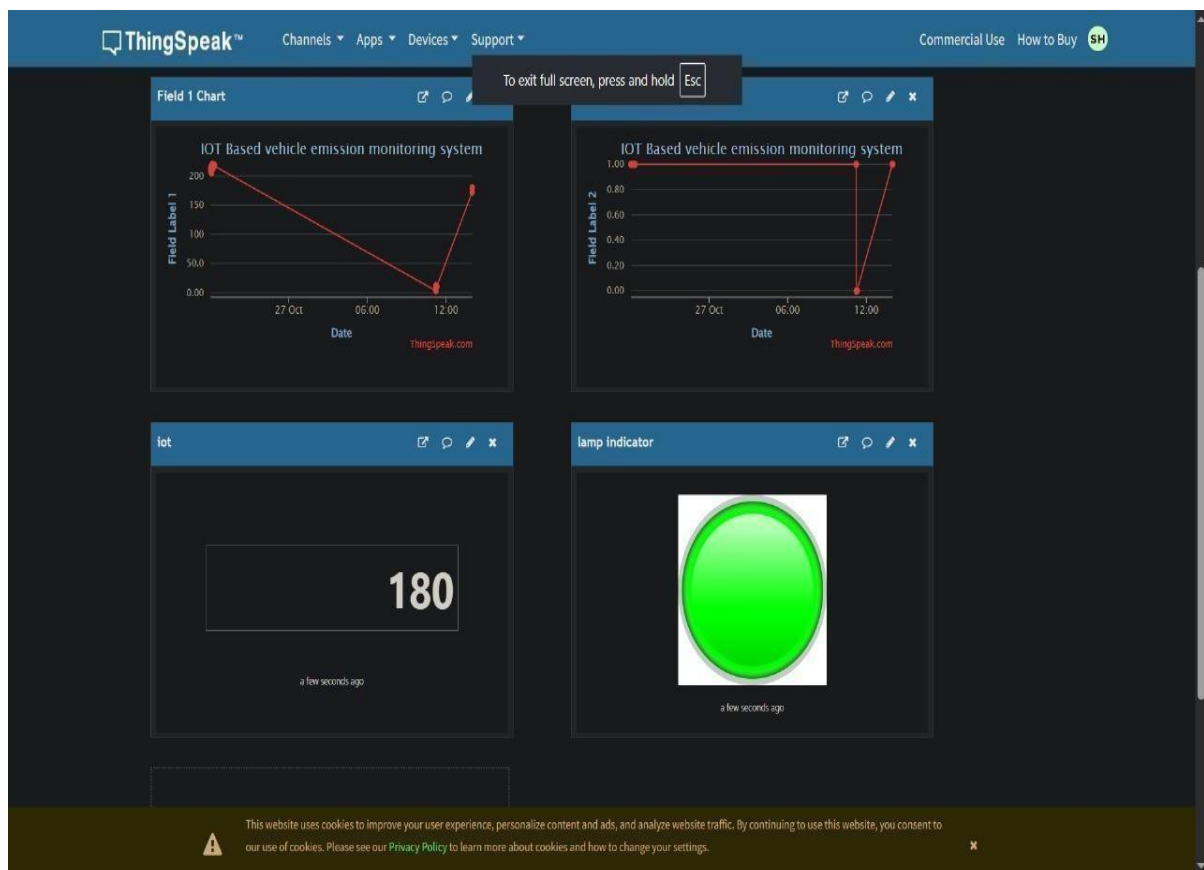


Figure 10.2: Software Output

CHAPTER 11

CONCLUSION AND FUTURE ENHANCEMENT

- The IoT vehicle emission system uses ESP8266, gas sensors, GPS, and GSM for realtime emission data and remote monitoring.
- Measurements upload to the cloud (like ThingSpeak) for alerts and emission trend analysis.
- Adding more sensors allows tracking of other pollutants such as particulates and nitrogen oxides.
- Secure data transfer and device authentication boost system safety and reliability.
- Mobile apps can give users live alerts and reminders about emissions and vehicle maintenance.
- Machine learning could help predict faults and forecast emission spikes.
- Large-scale use includes integration with national air quality systems and smart city platforms.
- Future improvements could add energy harvesting and further miniaturization for better efficiency.

REFERENCES

- [1] 1 S. T. Hasan, K. Rahman, and M. N. Islam, "IoT-Based Vehicle Exhaust Emission Monitoring and Notification System," *International Journal of Computer Applications*, vol. 183, no. 41, pp. 9-15, Feb. 2022.

- [2] 2 A. Jadhav, R. Bhujbal, and S. Patil, "Implementation of Real-Time Vehicle Pollution Monitoring System using Cloud and Gas Sensors," *IEEE International Conference on IoT and Embedded Systems (ICIIES)*, pp. 115-120, 2021.

- [3] 3 J. Wang, D. Hu, "Air Quality Analysis on ThingSpeak with Gas Sensors and ESP8266 Module," *Journal of Embedded Systems*, vol. 18, no. 2, pp. 57-64, 2023.

- [4] P. Kumar, M. Singh, "Real-Time Emission Monitoring Using MQ Sensors and Arduino ESP8266 Platform," *Sensors & Transducers*, vol. 255, no. 6, pp. 234-241, 2022.

- [5] R. Kumar and R. Sharma, "Design and Evaluation of an IoT-Based Emission Tracking Device with GSM and GPS," *International Journal of Advanced Computer Engineering*, vol. 13, no. 1, pp. 12-18, 2023.

[6]L. Smith, J. Fang, "Cloud-Connected Pollution Monitoring: A ThingSpeak Application for Smart Cities," Proceedings of the Smart Systems for Sustainable Cities Conference, 2022, pp. 63-70.

[7]N. Khosla, "Programming ESP8266 for Remote Environmental Analytics," Microcontroller Today, vol. 7, pp. 82-88, 2022.

[8] A. Patel, K. Oza, "Low-Cost Embedded System for Vehicle Emission Reporting and Control," IEEE International Conference on Environment and Electrical Engineering (EEEIC), pp. 420-425, 2021.

[9] D. S. Kumar, "Power Management Techniques for Automotive IoT Devices Using Buck Converters," Power and Embedded Electronics Journal, vol. 5, pp. 7783, 2020.Conference, 2015, pp. 1-5.

[10]D. L. Wu, Wing W. Y. NG, D. S. Yeung, and H. L. Ding, "A brief survey on current RFID applications," in Proc. International Conference on Machine Learning and Cybernetics, Baoding, July 12-15, 2016, pp. 2330-2334.